	SR UNIVERSITY Campus Warangal
	Program: II - B. Tech (CS&AI) Batch:
	Name: Ch. Lokesh Reg. No:2503A52324
	Department: Computer Science and AI Semester: I
	AI Assisted Coding-Assignment:13 Date:05-10-2026

Task 1

1. Question

Improve the readability of the legacy program by using meaningful variable names, creating calculate_average(), and improving formatting while preserving the result.

2. Prompt

Refactor this Python code for readability. Use meaningful variable names, create a function named calculate_average(), improve formatting, and preserve the original pass/fail behavior.

3. Code

Original code:

```
a=80
b=70
c=90
x=(a+b+c)/3
if x>=50:
   print("pass")
else:
   print("fail")
```

Refactored code:

```
def calculate_average(mark1, mark2, mark3):
   return (mark1 + mark2 + mark3) / 3


marks = [80, 70, 90]
average = calculate_average(*marks)


if average>= 50:
```

```
    print("pass")
else:
        print("fail")
```

4. Output

```
pass
```

5. Observation

The refactored code gives the same result as the original code. Meaningful names, a reusable function and consistent formatting make the program easier to understand.

Task 2

1. Question

Identify duplicated pass/fail logic, move it into a reusable function, test it with at least four students, and verify that the output remains correct.

2. Prompt

Identify the duplicated student pass/fail logic and refactor it into a reusable function. Test the function with four students and explain how removing duplication improves maintainability.

3. Code

Original code:

```
name1 ="Arun"
marks1 = 85
if marks1>= 50:
    print(name1, "Pass")
else:
    print(name1, "Fail")


name2 ="Priya"
marks2 = 72
if marks2>= 50:
    print(name2, "Pass")
else:
    print(name2, "Fail")
```

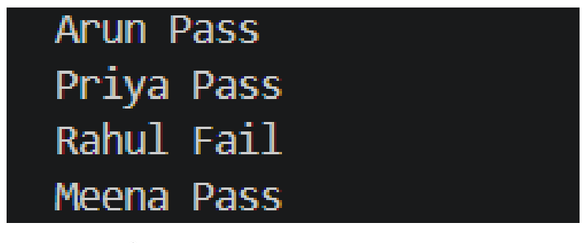
Refactored code:

```
def check_result(name, marks):
    if marks >= 50:
        print(name, "Pass")
    else:
```

```
        print(name, "Fail")


students = [("Arun", 85), ("Priya", 72), ("Rahul", 45), ("Meena", 52)]
for name, marks in students:
    check_result(name, marks)
```

4. Output



5. Observation

The repeated logic is placed in one reusable function. This reduces duplication and makes future changes easier because the pass/fail rule is maintained in one place.

Task 3

1. Question

Simplify the conditional logic by creating determine_priority(), test four age/temperature combinations, and compare the original and refactored behavior.

2. Prompt

Simplify the following priority condition without changing its behavior. Create determine_priority(age, temperature) and test four age/temperature combinations.

3. Code

Original code:

```
age = 68
temperature = 39
if age>= 60 and temperature>= 38:
    print("High Priority")
elif age>= 60 and temperature<38:
    print("Medium Priority")
elif age<60 and temperature>= 38:
    print("Medium Priority")
else:
    print("Normal Priority")
```

Refactored code:

```
def determine_priority(age, temperature):
    if age >= 60 and temperature >= 38:
```

```
    return "High Priority"
  if age >= 60 or temperature >= 38:
    return "Medium Priority"
  return "Normal Priority"


tests = [(68, 39), (68, 37), (40, 39), (40, 37)]
for age, temperature in tests:
  print(determine_priority(age, temperature))
```

4. Output

```
High Priority
Medium Priority
Medium Priority
Normal Priority
```

5. Observation

The repeated medium-priority cases are combined using OR. The four test cases produce the same behavior as the original conditional logic.

Task 4

1. Question

Improve readability and decide whether a set or dictionary is better for frequent patient-ID lookups. Implement the justified choice and test existing and missing IDs.

2. Prompt

Refactor the patient-ID lookup for readability. Explain whether a list or set is better for frequent membership checks, implement the suitable choice, and test an existing and a missing ID.

3. Code

Refactored code using a set for frequent membership lookup:

```
patient_ids = {101, 205, 309, 412, 518, 623}


def find_patient(patient_id):
  if patient_id in patient_ids:
    return "Patient Found"
  return "Patient Not Found"


print(find_patient(412))
print(find_patient(999))
```

4. Output

```
Patient Found
Patient Not Found
```

5. Observation

A set is suitable for frequent membership checks. Typical list membership is O(n), while average set membership is O(1). The refactored version is shorter and clearer for this lookup operation.

Task 5

1. Question

Identify at least five improvements, refactor using meaningful names and reusable functions, use suitable Python features for clarity, verify equivalent results, and record accepted changes.

2. Prompt

Review this legacy Python code and suggest at least five refactoring improvements. Use meaningful names, reusable functions and suitable Python features. Preserve the original results and explain the most useful change.

3. Code

Original code:

```
students = [("Arun", 78), ("Priya", 45), ("Rahul", 88), ("Meena", 52)]
total = 0
for s in students:
    total = total + s[1]
avg = total / len(students)
for s in students:
    if s[1] >= 50:
        print(s[0], "Pass")
    else:
        print(s[0], "Fail")
print("Average =", avg)
```

Refactored code:

```
def calculate_average(students):
    total_marks = sum(mark for name, mark in students)
    return total_marks / len(students)


def print_results(students):
    for name, mark in students:
```

```
        result = "Pass" if mark >= 50 else "Fail"
        print(name, result)

students = [("Arun", 78), ("Priya", 45), ("Rahul", 88), ("Meena", 52)]
print_results(students)
print("Average =", calculate_average(students))
```

4. Output

```
Arun Pass
Priya Fail
Rahul Pass
Meena Pass
Average = 65.75
```

5. Observation

Useful accepted improvements include meaningful names, reusable functions, sum(), tuple unpacking, a conditional expression, and separation of result printing from average calculation. The refactored program preserves the original results. The most useful change is separating the logic into functions because it improves readability and reuse.